DoT Front-end

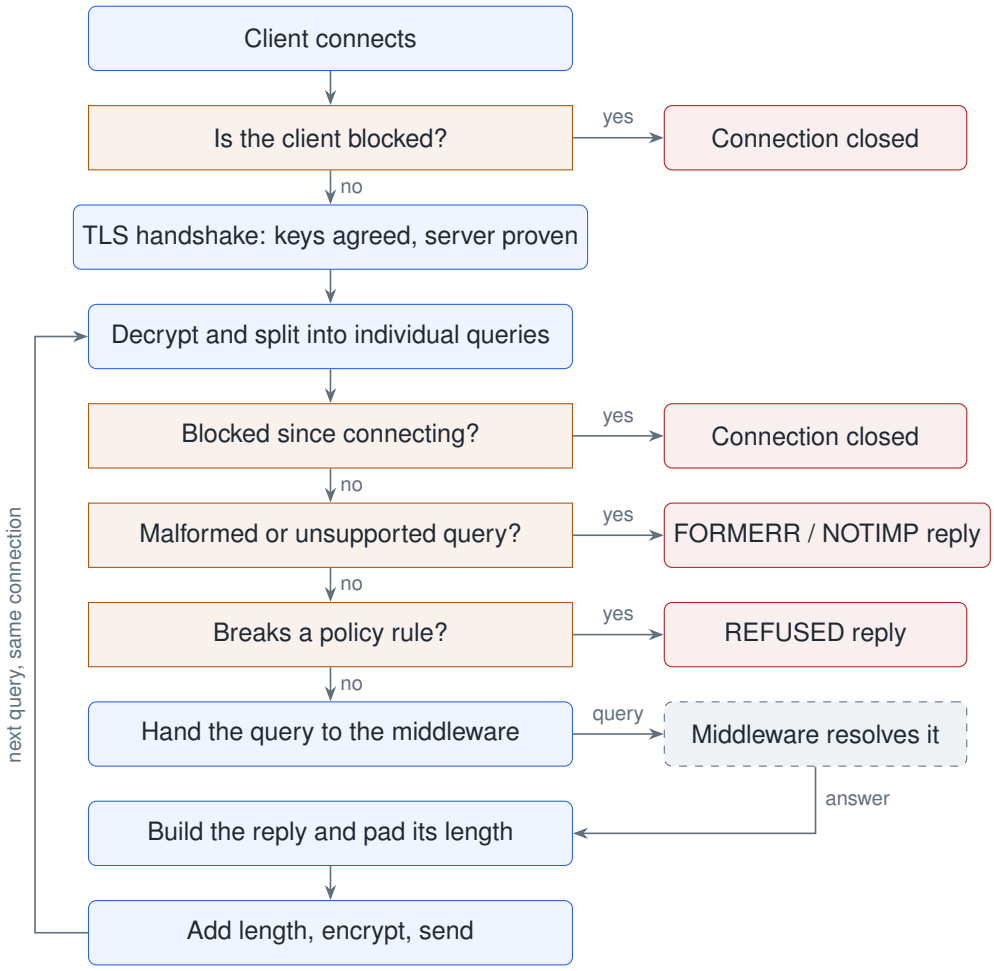
Design and progress

Owner: Dhruv · 20 October 2026

The front-end is the resolver's front door. It accepts encrypted DNS queries from clients over DNS-over-TLS (TCP port 853), checks each one, passes valid queries to the middleware, and sends back the encrypted answers. It also decides which clients and queries get rejected, and how.

We are implementing all of the features below. **5 of 11 are complete**; the rest are in progress.

How a query flows through the front-end



A connection stays open after a reply, so one client can send many queries without repeating the TLS handshake.

Features

Feature	What it does	Status
Query parsing and validation	Turns raw bytes into a checked query; rejects anything malformed or hostile without ever reading outside the message	**Done**
Name normalization	Treats `WWW.Example.COM` and `www.example.com.` as the same name, and rejects invalid names	**Done**
Reply building	Turns the middleware's answer into a correct DNS reply, always producing a valid message (falls back to SERVFAIL if needed)	**Done**
Length padding	Pads replies to a fixed block size so encrypted traffic doesn't reveal which site was looked up by its size	**Done**
Message framing	Splits the decrypted byte stream into individual queries, and prefixes each reply with its length	**Done**
TLS encryption	Encrypts every connection with OpenSSL; sets up keys securely and proves the server's identity	Remaining
Abuse blocking	Drops clients the middleware has flagged as abusive, before spending any work on them	Remaining
Policy rejections	Answers REFUSED for queries that break a rule, such as a blocked domain	Remaining
Connection handling	Serves many clients at once, several queries per connection, with timeouts and connection limits	Remaining
Configuration	Listen address, certificate and key paths, limits, blocked-domain list	Remaining
Runnable server	A server binary that can be tested end to end with `dig +tls`	Remaining

How the front-end rejects queries

Situation	Response	Why
Client flagged as abusive by the middleware	Connection closed, no reply	No work or bandwidth spent on attackers
Message too broken to read its ID, or not a query at all	Connection closed, no reply	There is nothing valid to reply to
Malformed query	FORMERR	Tells a buggy client what's wrong
Query kind we don't support	NOTIMP	A clear answer instead of silence
Query breaks a policy rule	REFUSED	A clear answer for legitimate clients
Middleware is overloaded	SERVFAIL	The client can retry later

What the response codes mean

Every DNS reply carries a response code. FORMERR, NOTIMP and REFUSED say the problem is with the query, so sending it again won't help; SERVFAIL says the problem is on the resolver's side, so retrying later can succeed.

Code	What it means in this resolver	Retry?
FORMERR Format error · code 1	**The query is broken.** It was readable enough to reply to, but breaks DNS rules: wrong section counts, a damaged or invalid name, a bad EDNS record, or extra bytes at the end	No
NOTIMP Not implemented · code 4	**Valid, but unsupported.** It asks for an operation other than a standard query, a class other than Internet (e.g. CHAOS), or a record type we don't handle yet (e.g. HTTPS)	No
REFUSED Refused · code 5	**Understood, but not allowed.** The query is valid and supported, but a policy rule forbids answering it, such as a blocked domain	No
SERVFAIL Server failure · code 2	**No answer could be produced.** The query was fine, but the middleware was overloaded, the upstream servers failed, or the answer couldn't be turned into a valid reply	Yes, later

How the encryption works

- **The session key is never sent.** Client and server each compute it from a key exchange (ECDHE), so someone watching the network can't obtain it.
- **The server proves its identity** by signing the handshake with its private key. The key never leaves the server, and the signature stops anyone impersonating it.
- **Forward secrecy.** A stolen private key would let an attacker impersonate the server in future, but not decrypt traffic recorded earlier.
- **Length padding** covers the one thing TLS doesn't hide: message sizes.
- **Key file protection.** The private key file is readable only by the server, and key files are kept out of git.

What the front-end needs from the middleware

The middleware is built separately by a teammate. The front-end relies on two things from it:

- **A way to submit a query.** The middleware either accepts it and answers later, rejects the client, or reports that it's overloaded.
- **A blocklist the front-end can check.** It answers "is this client blocked right now?"

Running the tests

```
make CXX=g++ test    # build and run all tests
make CXX=g++ asan    # same tests with memory-error detection
```